

Name:THEJASWINEE V S

Reg.no:711525BCS171

Batch:A

Project title: Warehouse Inventory System:

A warehouse system tracks inventory levels and product demand. Products must be sorted based on quantity and searched quickly. Implement using Heap Sort.

Course name:Data Structure

Complexity Level:Hard

Mini Project – Data Structures

Introduction

The Warehouse Inventory System is designed to efficiently manage product stock levels in a warehouse environment. The system stores product names and their quantities and allows users to organize the inventory based on available stock. In this project, the Heap Sort algorithm is used to sort products according to their quantities, ensuring efficient organization of inventory data. The system also allows users to search for a particular product and retrieve its stock information quickly. The implementation demonstrates the practical application of data structures such as arrays and algorithms like heap sort and linear search. This project helps improve understanding of sorting and searching techniques used in real-world inventory management systems. The program is implemented in Java and allows dynamic user input for adding products and displaying sorted inventory.

Problem Statement

A warehouse system tracks inventory levels and product demand. Products must be sorted based on quantity and searched quickly. The system should organize inventory data efficiently using appropriate data structures and sorting algorithms.

Objectives

- To store product inventory details in a structured format.
- To sort products based on quantity using Heap Sort.
- To search for products efficiently in the inventory.
- To demonstrate the practical use of data structures and algorithms.

Scope

The system can be used in warehouses or stores to manage product stock. It helps in organizing inventory records and retrieving product information quickly. The system can be further extended with database support, graphical interfaces, and advanced searching techniques.

Data Structures Used

Array

- Used to store product details such as product name and quantity.

Heap (Binary Heap)

- Used to implement heap sort for sorting products efficiently.

Algorithms Used

Heap Sort Algorithm

1. Read number of products.
2. Store product names and quantities.
3. Build a max heap from the quantity values.
4. Swap the first element with the last element.
5. Reduce heap size and apply heapify.
6. Repeat until the list is sorted

Pseudocode

Heap Sort

HEAPSORT(A, n)

BuildMaxHeap(A)

```
for i = n down to 2
    swap A[1] with A[i]
    reduce heap size
    heapify(A,1)
end for
```

Flowchart

You can draw this simple flowchart in your record:

Start

|

Read number of products

|

Input product name and quantity

|

Store in array

|

Apply Heap Sort

|

Display sorted products

|

Search product

|

Display result

|

End

Important Code Snippets

Heap Sort Function

```
static void heapSort(Product arr[], int n) {  
    for (int i = n/2 - 1; i >= 0; i--)  
        heapify(arr, n, i);  
  
    for (int i = n-1; i > 0; i--) {  
        Product temp = arr[0];  
        arr[0] = arr[i];  
        arr[i] = temp;  
  
        heapify(arr, i, 0);  
    }  
}
```

Explanation

The heapSort() function first builds a max heap from the product array. Then it repeatedly swaps the root element with the last element and reduces the heap size. Heapify is applied again to maintain the heap property until the array becomes fully sorted.

Sample Input / Output

Example Output:

Enter number of products: 3

Enter product name: Rice

Enter quantity: 120

Enter product name: Sugar

Enter quantity: 80

Enter product name: Wheat

Enter quantity: 150

Products sorted by quantity:

Sugar - 80

Rice - 120
Wheat - 150

Enter product to search: Rice

Product Found!
Quantity: 120

Test Cases

Test Case	Input	Expected Output
Normal Case	Multiple products	Sorted list displayed
Edge Case	Single product	Displays same product
Duplicate Case	Same quantity values	Sorted correctly
Search Case	Valid product	Product found
Invalid Case	Non-existing product	Product not found

Time and Space Complexity

Operation	Complexity
Heap Sort	$O(n \log n)$
Insertion	$O(1)$
Search	$O(n)$
Space Complexity	$O(1)$

References

1. Introduction to Algorithms – Thomas H. Cormen
2. Data Structures Using C – Reema Thareja
3. NPTEL Data Structures Course
4. Coursera – Data Structures and Algorithms Specialization

Source code

```
import java.util.Scanner;

class Product {
    String name;
    int quantity;

    Product(String name, int quantity) {
        this.name = name;
        this.quantity = quantity;
    }
}

public class WarehouseInventory {

    static void heapify(Product arr[], int n, int i) {
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left].quantity > arr[largest].quantity)
            largest = left;

        if (right < n && arr[right].quantity > arr[largest].quantity)
            largest = right;

        if (largest != i) {
            Product temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;
            heapify(arr, n, largest);
        }
    }

    static void heapSort(Product arr[], int n) {
        for (int i = n / 2 - 1; i >= 0; i--)
```

```
        heapify(arr, n, i);
    for (int i = n - 1; i >= 0; i--) {
        Product temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}

static void display(Product arr[], int n) {
    System.out.println("\nProducts sorted by quantity:");
    for (int i = 0; i < n; i++) {
        System.out.println(arr[i].name + " - " + arr[i].quantity);
    }
}

static void search(Product arr[], int n, String key)
{
    boolean found = false;
    for (int i = 0; i < n; i++) {
        if (arr[i].name.equalsIgnoreCase(key)) {
            System.out.println("Product Found!");
            System.out.println("Quantity: " + arr[i].quantity);
            found = true;
            break;
        }
    }
    if (!found) {
        System.out.println("Product not found.");
    }
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    System.out.print("Enter number of products: ");  
    int n = sc.nextInt();  
    sc.nextLine();  
    Product arr[] = new Product[n];  
    for (int i = 0; i < n; i++)  
    {  
        System.out.print("Enter product name: ");  
        String name = sc.nextLine();  
        System.out.print("Enter quantity: ");  
        int qty = sc.nextInt();  
        sc.nextLine();  
        arr[i] = new Product(name, qty);  
    }  
    heapSort(arr, n);  
    display(arr, n);  
    System.out.print("\nEnter product name to search: ");  
    String key = sc.nextLine();  
    search(arr, n, key);  
    sc.close();  
}  
}
```

Conclusion

The Warehouse Inventory System was successfully implemented using appropriate data structures and algorithms. The project demonstrated how inventory data can be efficiently managed by storing product information and organizing it based on quantity. The Heap Sort algorithm was used to sort the products effectively, which ensures better performance compared to simple sorting techniques for larger datasets. Searching functionality was also implemented to quickly locate a specific product in the inventory. Through this project, the practical application of arrays, heap data structures, sorting, and searching algorithms was clearly understood. Overall, the system provides a simple and efficient way to manage warehouse inventory and highlights the importance of data structures in solving real-world problems.